

Telegram Aggregator — Bot + Mini App:

техническая спецификация

Платформа-агрегатор рабочих систем внутри Telegram:
уведомления, управление задачами и единая лента поверх
нескольких систем. **Phase 1** **стартует сразу с двух провайдеров — ClickUp и Wrike** — это форсирует честную абстракцию адаптеров с
первого дня: ядро не знает ничего провайдер-специфичного.

1. Обзор

1.1 Проблема

Многие рабочие системы (ClickUp, Wrike, далее — другие таск-трекеры, CRM, тикет-системы) не имеют нативной интеграции с Telegram. Команды, для которых Telegram — основной канал коммуникации, вынуждены жить на email-уведомлениях и ручных проверках.

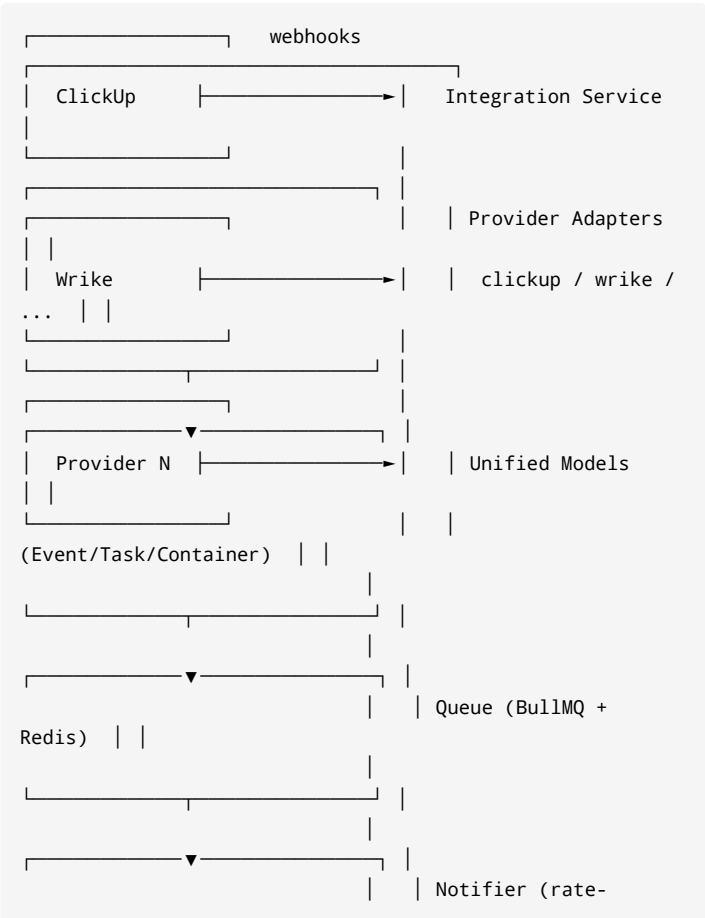
1.2 Решение

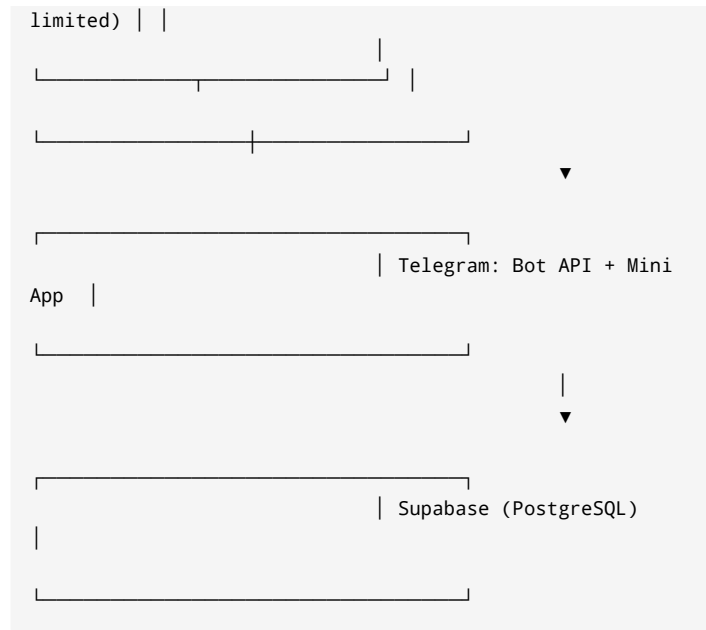
Компонент	Роль
Integration Service (Node.js/TypeScript)	Приём вебхуков от провайдеров, нормализация событий, доставка в Telegram
Telegram Bot	Уведомления + быстрые команды (создать задачу, сменить статус)
Telegram Mini App (React + TypeScript)	Полноценный UI: подключение провайдеров, настройка подписок, unified inbox

1.3 Режимы

Режим	Направление	Приоритет
Notifications	Provider → Telegram	P0
Task Bot / Mini App	Telegram → Provider	P1

2. Архитектура





Железное правило: ядро (webhook endpoint, worker, notifier, бот, Mini App) оперирует только unified-моделями. Провайдер-специфичный код живёт исключительно внутри адаптера. Если для новой фичи ядру нужно знать имя провайдера — интерфейс адаптера спроектирован неправильно.

2.1 Интерфейс ProviderAdapter

Интерфейс спроектирован под пересечение реальных различий ClickUp и Wrike — это гарантирует, что он выдержит третьего провайдера:

```
// Форма credentials известна только адаптеру. Это НЕ строка-токен:
// ClickUp/Wrike — { token }, Jira — { baseUrl, email, apiToken }.
// Ядро хранит и передаёт credentials как opaque-объект.
type ProviderCredentials = Record<string, string>

// Дескриптор возможностей: ядро адаптирует поведение
```

```

// (UI онбординга, refresh-джобы, skip enrich) без if
(provider === ...).
interface AdapterCapabilities {
  webhookSetup: 'auto' | 'admin-required'; // Jira Basic
  auth: только админ
  webhookLifetimeDays?: number;           // Jira OAuth:
  30 → нужен refresh-job
  payload: 'rich' | 'minimal';           // rich (Jira)
  → enrichEvent = no-op
}

interface ProviderAdapter {
  readonly id: string;                     // 'clickup' |
  'wrike' | 'jira' | ...
  capabilities(): AdapterCapabilities;

  // — Вебхуки

  // Некоторые провайдеры (Wrike) требуют handshake при
  регистрации
  // вебхука: null → обычная нотификация, иначе — ответ
  на handshake.
  handleHandshake?(headers: Headers): HandshakeResponse |
  null;

  // Верификация подписи. rawBody обязателен: HMAC
  считается
  // от исходных байтов, а не от re-serialized JSON.
  verifyWebhook(rawBody: Buffer, headers: Headers, secret:
  string): boolean;

  // ВСЕГДА массив: Wrike батчит несколько событий в
  запрос;
  // Jira из одного issue_updated с changelog порождает
  несколько
  // unified-событий (статус + ассайни + дедлайн). Headers
  нужны:
  // Jira передаёт id доставки в X-Atlassian-Webhook-
  Identifier.
  parseEvents(payload: unknown, headers: Headers):
  UnifiedEvent[];

  // Догрузка данных. У ClickUp/Wrike payload минимален →

```

```

fetch задачи;
  // у Jira payload содержит issue целиком → возврат
  события как есть.
  enrichEvent(event: UnifiedEvent, creds:
ProviderCredentials): Promise<UnifiedEvent>;

  // Регистрация вебхука; источник секрета инкапсулирован
  // (ClickUp выдаёт сам, Wrike/Jira – генерируем мы).
  registerWebhook(creds: ProviderCredentials, scope:
WebhookScope): Promise<WebhookRef>;
  refreshWebhook?(creds: ProviderCredentials, ref:
WebhookRef): Promise<void>;
  deleteWebhook(creds: ProviderCredentials,
providerWebhookId: string): Promise<void>;

  // — Действия (бот / Mini App)
  

---


  verifyCredentials(creds: ProviderCredentials):
Promise<AccountInfo>;
  createTask(creds: ProviderCredentials, input:
CreateTaskInput): Promise<TaskRef>;
  updateTask(creds: ProviderCredentials, taskId: string,
patch: TaskPatch): Promise<void>;
  setStatus(creds: ProviderCredentials, taskId: string,
statusId: string): Promise<void>;
  addComment(creds: ProviderCredentials, taskId: string,
text: string): Promise<void>;
  getTask(creds: ProviderCredentials, taskId: string):
Promise<UnifiedTask>;

  // — Навигация (unified-иерархия)
  

---


  // ClickUp: team→space→folder→list; Wrike:
account→space→folder;
  // Jira: site→project. Ядро видит одно дерево
  Container.
  listContainers(creds: ProviderCredentials, parentId?:
string): Promise<Container[]>;

  // Статусы запрашиваются ПО ЗАДАЧЕ, а не по контейнеру:
  // в Jira статус меняется только через доступные
  транзишены workflow,
  // и их набор зависит от текущего состояния конкретной

```

```

задачи.
    // Для ClickUp/Wrike вырождается в статусы списка/
воркфлоу.
    getAvailableStatuses(creds: ProviderCredentials, taskId:
string): Promise<StatusDef[]>;

    // — Rate limiting
    _____

    rateLimit(connection: Connection): RateLimitConfig; //
фикс. rpm или Retry-After-driven
}

```

2.2 Unified-модели

```

type UnifiedEventType =
    | 'task.created' | 'task.updated' | 'task.deleted'
    | 'task.assigned' | 'task.status_changed' |
'task.due_changed'
    | 'comment.added';

interface UnifiedEvent {
    provider: string;
    eventType: UnifiedEventType;
    dedupeKey: string;           // правило построения
– у адаптера (§4)
    taskId: string;
    taskName?: string;         // может отсутствовать
до enrich
    taskUrl?: string;
    actor?: string;
    details: Record<string, unknown>; // old/new status,
assignees, comment preview...
    raw: unknown;
}

// Единая иерархия навигации: адаптер отвечает за маппинг
// своего дерева в это представление.
interface Container {
    id: string;
    name: string;
    kind: 'root' | 'space' | 'folder' | 'tasklist';
}

```

```

    canContainTasks: boolean;           // ClickUp: только
list; Wrike: любой folder
    parentId?: string;
}

// Статусы: у обоих провайдеров кастомные воркфлоу; ядро
оперирует
// категориями, адаптер маппит категорию ↔ конкретный
статус.
interface StatusDef {
    id: string;                         // provider-specific
id
    name: string;
    category: 'open' | 'in_progress' | 'done' | 'cancelled';
}

interface UnifiedTask {
    provider: string;
    id: string;
    name: string;
    description?: string;
    status: StatusDef;
    assignees: string[];
    dueDate?: string;
    url: string;
    containerId: string;
}

```

Inline-кнопка «Готово» под уведомлением работает так: ядро вызывает `getAvailableStatuses(taskId)`, находит статус с `category: 'done'`, вызывает `setStatus`. Для ClickUp это запись поля `status`, для Wrike — `customStatus`, для Jira — выполнение транзишена. Ни одной строчки провайдер-специфичного кода в ядре.

2.3 Стек

Слой	Технология	Комментарий
Runtime	Node.js 22 LTS	

Слой	Технология	Комментарий
Язык	TypeScript	весь стек, включая бота
Framework	Fastify 5.x	HTTP-сервер + вебхуки
Telegram SDK	grammY	лучшая TS-типизация; telegraf — допустимая альтернатива
Mini App	React + TypeScript + Vite, @telegram-apps/sdk-react	
БД	Supabase (PostgreSQL)	
Очередь	BullMQ + Redis — обязательно с Phase 1	см. §5

3. Схема БД (Supabase, provider-agnostic)

```
-- Пользователи (идентичность = Telegram)
create table users (
  id uuid primary key default gen_random_uuid(),
  telegram_id bigint not null unique,
  telegram_username text,
  first_name text,
  last_name text,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

-- Подключения к провайдерам
create table provider_connections (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references users(id) on delete
  cascade,
  provider text not null,          -- 'clickup',
  'wrike', 'jira', ...
```



```

-- Оpaque credentials: JSON, зашифрованный AES-256-GCM в
приложении.
-- Форму знает только адаптер: ClickUp/Wrike – {token},
-- Jira – {baseUrl, email, apiToken}. НЕ одна строка-
токен.
credentials text not null,
scope_id text not null,          -- ClickUp:
team_id; Wrike: account_id; Jira: site url
is_active boolean default true,
created_at timestampz default now(),
updated_at timestampz default now(),
unique(user_id, provider, scope_id)
);

-- Зарегистрированные вебхуки у провайдеров.
-- Секрет per-webhook: у ClickUp его выдаёт провайдер при
создании,
-- у Wrike его генерируем мы и передаём при регистрации –
-- для ядра разница инкапсулирована в registerWebhook.
create table webhooks (
  id uuid primary key default gen_random_uuid(),
  connection_id uuid not null references
provider_connections(id) on delete cascade,
  provider text not null,
  provider_webhook_id text not null,
  secret text not null,          -- AES-256-GCM
at rest
  scope jsonb default '{}',      --
account/space/folder/JQL scope вебхука
  expires_at timestampz,         -- для
провайдеров с TTL вебхуков
                                -- (Jira OAuth:
30 дней) – refresh-job
  created_at timestampz default now(),
  unique(provider, provider_webhook_id)
);

-- Алиасы контейнеров для быстрого доступа из бота
create table mappings (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references users(id) on delete
cascade,
  provider text not null,

```

```

    alias text not null,                -- "dev", "Dev
Tasks"
    container_id text not null,         -- ClickUp:
list_id; Wrike: folder_id
    container_meta jsonb default '{}',
    is_default boolean default false,
    created_at timestampz default now(),
    unique(user_id, provider, alias)
);

-- Подписки чатов на уведомления
create table subscriptions (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references users(id) on delete
cascade,
    connection_id uuid not null references
provider_connections(id) on delete cascade,
    telegram_chat_id bigint not null,
    filters jsonb default '{}',        -- container-
филтры, provider-specific внутри jsonb
    event_types text[] not null default

'{task.created,task.updated,task.assigned,task.status_chan
ged,task.due_changed,comment.added}',
    is_active boolean default true,
    created_at timestampz default now(),
    -- защита от дублей уведомлений при повторном /subscribe
    unique(connection_id, telegram_chat_id)
);

-- Лог доставки + идемпотентность
create table notification_log (
    id uuid primary key default gen_random_uuid(),
    subscription_id uuid not null references
subscriptions(id) on delete cascade,
    dedupe_key text not null,
    event_type text not null,
    delivered_at timestampz default now(),
    telegram_message_id bigint,
    unique(subscription_id, dedupe_key)    --
идемпотентность на уровне БД
);

```

Заметьте: в схеме нет ни одной ClickUp- или Wrike-специфичной колонки — всё провайдер-зависимое живёт в `provider` + `jsonb` - полях.

3.1 Изоляция данных и RLS

Пользователи аутентифицируются через Telegram, а не через Supabase Auth, поэтому классические RLS-политики на `auth.uid()` здесь неприменимы. Сервис ходит в БД с `service key` (обходит RLS). Подход:

1. **Базовый уровень** — изоляция в приложении: каждый запрос сервиса фильтруется по `user_id`.
2. **Для Mini App** — честный RLS: бэкэнд валидирует `initData`, выпускает Supabase-совместимый JWT с claim `telegram_id`, RLS-политики пишутся через этот claim. Тогда Mini App может ходить в Supabase напрямую (PostgREST/Realtime).

3.2 Шифрование токенов

- AES-256-GCM **только на стороне приложения** (`node:crypto`), уникальный IV на запись, формат хранения `iv:tag:ciphertext` (base64).
- Ключ — в `ENCRYPTION_KEY` (32 байта hex), в БД никогда не передаётся.

4. Провайдер-адаптеры

4.1 ClickUp (API v2)

Токен: personal token, верификация через `GET /api/v2/user`.
`scope_id` = `team_id`.

Вебхуки. Подпись: заголовок `X-Signature`, HMAC-SHA256 от сырого тела; секрет уникален per-webhook и **возвращается провайдером** в ответе на `POST /team/{team_id}/webhook` —

сохраняется в `webhooks`. Handshake не требуется
(`handleHandshake` → всегда null).

Payload — одно событие на запрос:

```
{
  "webhook_id": "...",
  "event": "taskStatusUpdated",
  "task_id": "abc123",
  "history_items": [ { "id": "...", "before": {...},
    "after": {...}, "user": {...} } ]
}
```

- В payload нет `team_id` и данных задачи → матчинг подписок по `webhook_id`, форматирование после `enrichEvent` (`GET /task/{task_id}`).
- **dedupeKey** = `event + ':' + history_items[].id` (стабильные id записей истории; fallback — sha256 тела).

Маппинг событий:

ClickUp	Unified
<code>taskCreated</code>	<code>task.created</code>
<code>taskUpdated</code>	<code>task.updated</code>
<code>taskDeleted</code>	<code>task.deleted</code>
<code>taskAssigneeUpdated</code>	<code>task.assigned</code>
<code>taskStatusUpdated</code>	<code>task.status_changed</code>
<code>taskDueDateUpdated</code>	<code>task.due_changed</code>
<code>taskCommentPosted</code> / <code>taskCommentUpdated</code>	<code>comment.added</code>

Навигация (`listContainers`): `team` → `space` (`GET /team/{id}/space`) → `folder` (`GET /space/{id}/folder`) + `folderless-`

списки (GET /space/{id}/list возвращает **только их**) → списки папок (GET /folder/{id}/list). canContainTasks: true только у tasklist.

Действия: создание — POST /list/{list_id}/task; статус — PUT /task/{id} с полем status; ассайни — assignees: { add: [id], rem: [id] }; комментарий — POST /task/{id}/comment.

Rate limit: зависит от плана (Free/Unlimited/Business: 100 rpm per token; Business Plus: 1 000; Enterprise: 10 000) — отдаётся ядру через rateLimit().

4.2 Wrike (API v4)

Токен: permanent access token (создаётся в Wrike → Apps & Integrations) или OAuth 2.0; для MVP — permanent token, верификация через GET /api/v4/contacts?me=true. scope_id = account_id.

Вебхуки. Регистрация: POST /api/v4/webhooks (account-scope) либо POST /spaces/{id}/webhooks / POST /folders/{id}/webhooks для узких скоупов; параметры — hookUrl и secret (**секрет генерируем мы**, храним в webhooks). Скоуп пишется в webhooks.scope.

Два отличия от ClickUp, инкапсулированные адаптером:

1. **Handshake:** при регистрации Wrike присылает запрос с заголовком X-Hook-Secret, который нужно эхом вернуть в ответе — это реализует handleHandshake.
2. **Батчинг:** нотификации приходят **массивом событий** в одном запросе — поэтому parseEvents в интерфейсе всегда возвращает массив.

Подпись рабочих нотификаций: заголовок X-Hook-Secret = HMAC-SHA256 от тела, ключ — наш секрет (verifyWebhook).

Payload:

```
[
  {
    "webhookId": "IEACW...",
    "eventType": "TaskStatusChanged",
    "taskId": "IEAAA...",
    "eventAuthorId": "KUAA...",
    "oldCustomStatusId": "...",
    "customStatusId": "...",
    "lastUpdatedDate": "2026-07-01T10:00:00Z"
  }
]
```

- Данных задачи в payload нет → `enrichEvent` через `GET /api/v4/tasks/{taskId}`.
- Уникального id события нет → `dedupeKey` = `eventType + ':' + taskId + ':' + lastUpdatedDate` (+ status ids для событий статуса; fallback — sha256 элемента).

Маппинг событий:

Wrike	Unified
TaskCreated	task.created
TaskDeleted	task.deleted
TaskTitleChanged, TaskDescriptionChanged, TaskImportanceChanged	task.updated
TaskResponsiblesAdded / TaskResponsiblesRemoved	task.assigned
TaskStatusChanged	task.status_changed
TaskDatesChanged	task.due_changed
CommentAdded	comment.added

Навигация (listContainers): account → spaces (GET /spaces) → folders/projects (GET /spaces/{id}/folders , дерево через поле childIds). В отличие от ClickUp, задачи живут в любом фолдере → canContainTasks: true у всех folder .

Статусы: Wrike использует кастомные воркфлоу (GET /workflows); адаптер маппит customStatuses[].group (Active/Completed/Deferred/Cancelled) на unified-категории in_progress/done/open/cancelled .

Действия: создание — POST /folders/{folderId}/tasks ; обновление/статус — PUT /tasks/{id} (customStatus); комментарий — POST /tasks/{id}/comments .

Rate limit: динамический, зависит от плана аккаунта; консервативный дефолт задаётся конфигом и переопределяется per-connection через rateLimit() .

4.3 Jira Cloud (REST API v3)

Jira спроектирована иначе, чем ClickUp/Wrike, и именно она диктует три ключевых решения интерфейса (§2.1): opaque credentials, getAvailableStatuses(taskId) и capabilities() .

Credentials: не единый токен, а тройка { baseUrl: https://{site}.atlassian.net, email, apiToken } — Basic auth (base64(email:apiToken)). scope_id = URL сайта. В перспективе — OAuth 2.0 (3LO) с роутингом через api.atlassian.com/ex/jira/{cloudId} ; форма credentials меняется, но для ядра это по-прежнему opaque-объект.

Вебхуки — два режима:

Режим	Кто создаёт	TTL	Подпись
Admin webhooks (UI / admin REST)	только администратор Jira	бессрочно	HMAC-SHA256, наш секрет

Режим	Кто создаёт	TTL	Подпись
Dynamic webhooks (POST /rest/api/3/webhook)	только OAuth 2.0- приложение	30 дней, продление через refresh	bearer- механизм приложения

Для MVP (Basic auth) → admin webhooks:

`capabilities().webhookSetup = 'admin-required'`, Mini App при подключении Jira показывает пошаговую инструкцию для администратора (URL + сгенерированный секрет + список событий + опциональный JQL-фильтр). При переходе на OAuth → dynamic webhooks + repeatable-job продления по `webhooks.expires_at`.

Подпись: секрет задаём мы при регистрации; Jira присылает HMAC-SHA256 от сырого тела в заголовке `X-Hub-Signature` в формате `sha256=<hex>` (стандарт WebSub) — ложится в существующий `verifyWebhook` без изменений. Секрет невозможно получить из Jira после создания — только хранить у себя (таблица `webhooks`).

Payload — богатый (`capabilities().payload = 'rich'`): содержит issue целиком, `user` и `changelog` → `enrichEvent` возвращает событие как есть, дополнительный fetch не нужен. Экономия rate limit по сравнению с ClickUp/Wrike.

События и их разворачивание:

Jira	Unified
<code>jira:issue_created</code>	<code>task.created</code>
<code>jira:issue_deleted</code>	<code>task.deleted</code>
<code>jira:issue_updated</code> + <code>changelog.items[]</code>	1..N событий: <code>item status</code> → <code>task.status_changed</code> ; <code>assignee</code> → <code>task.assigned</code> ; <code>duedate</code> →

Jira	Unified
	<code>task.due_changed</code> ; прочие → <code>task.updated</code>
<code>comment_created</code>	<code>comment.added</code>

Один запрос Jira может породить несколько unified-событий — сигнатура `parseEvents(...): UnifiedEvent[]` покрывает это так же, как батчи Wrike.

dedupeKey: заголовок `X-Atlassian-Webhook-Identifier` (id доставки; Jira ретраит доставки) + индекс `changelog-item` — поэтому `parseEvents` принимает `headers`.

Статусы — только транзишены: статус нельзя записать напрямую; `GET /rest/api/3/issue/{key}/transitions` возвращает переходы, доступные из текущего состояния конкретной задачи, смена — `POST .../transitions` с id перехода. Отсюда сигнатура `getAvailableStatuses(taskId)`; маппинг на категории — через `statusCategory` (`new` → `open`, `indeterminate` → `in_progress`, `done` → `done`).

Контейнеры: `site` → `project` (`GET /rest/api/3/project/search`); задачи живут в проектах (`canContainTasks: true`). Issue keys (`PROJ-123`) удобны для `/task` в боте.

Форматы текста: описания и комментарии — ADF (Atlassian Document Format, JSON-дерево, не plain text). `addComment` конвертирует текст → минимальный ADF-документ; `getTask` рендерит ADF → plain/HTML для карточки. Инкапсулировано в адаптере, `UnifiedTask.description` — всегда текст.

Rate limit: динамический, без фиксированного `rpm` — стратегия «консервативный дефолт + обязательное соблюдение `Retry-After` на 429» (интерфейс `rateLimit()` это поддерживает).

Действия: создание — `POST /rest/api/3/issue`; обновление — `PUT /rest/api/3/issue/{key}`; комментарий — `POST /rest/api/3/issue/{key}/comment` (ADF).

4.4 Чек-лист для добавления провайдера N

Новый провайдер = один файл-адаптер + строка в реестре.

Адаптер обязан ответить на вопросы:

1. Какова форма credentials (один токен? токен + email + base URL? OAuth?) и как они верифицируются?
 2. Кто может зарегистрировать вебхук — сам пользователь через API или только администратор (`webhookSetup`)? Есть ли у вебхука TTL и нужен ли refresh-job?
 3. Как верифицируется вебхук (подпись? handshake? и то и другое?) и откуда берётся секрет — выдаёт провайдер или генерируем мы?
 4. Одно событие на запрос, батч или одно провайдер-событие → несколько unified (changelog)?
 5. Что есть в payload (`rich` / `minimal`) → нужен ли fetch в `enrichEvent` ?
 6. Из чего строится стабильный `dedupeKey` (id события, заголовок доставки, timestamp)?
 7. Как дерево провайдера ложится на `Container` и где живут задачи?
 8. Статусы пишутся напрямую или через транзишены? Как ложатся на 4 unified-категории?
 9. Какой rate limit — фиксированный per-plan или динамический через `Retry-After` ?
 10. Какой формат текста (plain / markdown / ADF) и как он конвертируется в обе стороны?
-

5. Обработка вебхуков

Оба провайдера деактивируют вебхук после серии неуспешных/медленных ответов, поэтому HTTP-хендлер отвечает мгновенно, а обработка идёт через очередь. Endpoint один, параметризованный:

```
POST /webhooks/:provider
1. adapter = registry.get(params.provider)
2. Если adapter.handleHandshake(headers) вернул ответ –
вернуть его и выйти
(регистрация вебхука Wrike)
3. Прочитать сырое тело (raw body – HMAC от исходных
байтов)
4. Найти секрет по webhook id из payload/заголовков →
adapter.verifyWebhook
5. Положить job в BullMQ (payload как есть)
6. Немедленно ответить 200
```

Worker (асинхронно):

```
7. events = adapter.parseEvents(payload, headers) //
1..N событий
(батчи Wrike, разворачивание changelog Jira, headers
для dedupe)
8. Для каждого события:
а. adapter.enrichEvent – для 'minimal'-провайдеров
GET задачи через
лимитер credentials владельца connection; для
'rich' (Jira) – no-op
б. Найти активные подписки connection (+ container-
фильтры из task-данных)
в. Для каждой подписки: INSERT в notification_log
ON CONFLICT (subscription_id, dedupe_key) DO
NOTHING
→ при конфликте пропустить (идемпотентность
гарантирована БД)
г. Рендер шаблона из UnifiedEvent → отправка через
Telegram-лимитер
д. Записать telegram_message_id
9. Ретраи BullMQ с экспоненциальным backoff; DLQ после N
попыток
```

6. Telegram: бот, лимиты, форматирование

6.1 Лимиты Bot API

- ~30 сообщений/сек суммарно по всем чатам;
- ~1 сообщение/сек в один и тот же приватный чат;
- **20 сообщений/мин в одну группу** — самое чувствительное для командных чатов.

Notifier: per-chat token bucket + глобальный лимитер; при всплеске событий в одну группу — схлопывание уведомлений в дайджест (batching).

6.2 Форматирование сообщений

HTML parse mode (экранируются только `<`, `>`, `&`) — legacy

Markdown ломается на спецсимволах в названиях задач.

Шаблоны рендерятся из `UnifiedEvent` и одинаковы для всех провайдеров (различается только бейдж источника):

Unified event	Шаблон
<code>task.created</code>	<code>{{provider}}</code> Новая задача → <code>{name}</code> , <code>{container}</code> , ссылка
<code>task.assigned</code>	<code>{{provider}}</code> <code>{name}</code> → <code>{assignee}</code> , от <code>{actor}</code>
<code>task.status_changed</code>	<code>{{provider}}</code> <code>{name}</code> : <code>{old}</code> → <code>{new}</code>
<code>task.due_changed</code>	<code>{{provider}}</code> <code>{name}</code> : новый дедлайн <code>{date}</code>
<code>comment.added</code>	<code>{{provider}}</code> Комментарий к <code>{name}</code> от <code>{author}</code> : <code>{preview}</code>

6.3 Вебхук бота

`setWebhook(..., { secret_token })`; Telegram присылает его в заголовке `X-Telegram-Bot-API-Secret-Token` — сверять на каждом апдейте. Режим `polling` — для локальной разработки (`BOT_MODE=polling`).

6.4 Команды

Все команды провайдер-агностичны; при нескольких подключениях провайдер выбирается алиасом (`#dev` знает свой провайдер) или inline-кнопками.

Команда	Аргументы	Действие
<code>/start</code>	—	Приветствие + кнопка открытия Mini App
<code>/connect</code>	<code><provider></code> <code><token></code>	<code>adapter.verifyCredentials</code> , сохранение; сообщение пользователя удаляется (<code>deleteMessage</code>); поддерживает только «простых» провайдеров (один токен) — для Jira путь один: Mini App
<code>/newtask</code>	<code><название></code> <code>[#alias]</code>	Создание задачи в контейнере по алиасу (или default)
<code>/task</code>	<code><id></code>	Краткая карточка (<code>getTask</code>) + inline-кнопки
<code>/comment</code>	<code><id></code> <code><текст></code>	Комментарий
<code>/status</code>	<code><id></code> <code><статус></code>	Смена статуса (через <code>getStatuses</code> + маппинг категорий)
<code>/assign</code>	<code><id></code> <code>@user</code>	Назначение
<code>/due</code>	<code><id></code> <code><дата></code>	Дедлайн

Команда	Аргументы	Действие
<code>/browse</code> , <code>/map</code>		Навигация по <code>listContainers</code> + алиасы
<code>/subscribe</code> , <code>/unsubscribe</code>		Подписка чата (идемпотентна)
<code>/help</code>	—	Справка

Inline-кнопки под карточкой задачи и уведомлениями: `[Взять в работу]` `[Готово]` `[Комментировать]` — работают через unified-категории статусов (§2.2). Многошаговые сценарии не строятся в чате — уводятся в Mini App через `web_app`-кнопку.

7. Telegram Mini App

7.1 Стек

React 18 + TypeScript + Vite, `@telegram-apps/sdk-react` (тема, `viewport`, `BackButton`, `MainButton`), TanStack Query. Статика раздаётся с того же VPS через nginx (обязателен HTTPS — требование Telegram).

7.2 Аутентификация

1. Mini App получает `initData` из Telegram.
2. Бэкенд валидирует подпись: HMAC-SHA256, ключ = `HMAC_SHA256("WebAppData", BOT_TOKEN)`; проверяет `auth_date` (окно, напр., 1 час).
3. Бэкенд выпускает короткоживущий JWT (Supabase-совместимый — тогда работает RLS, §3.1).

7.3 Экраны (MVP)

- **Connections** — подключение/отключение провайдеров, ввод `credentials` (не через чат; форма полей — из адаптера: один

токен у ClickUp/Wrike, site + email + API token у Jira), выбор workspace; при `webhookSetup: 'admin-required'` (Jira) — пошаговая инструкция для администратора с URL и сгенерированным секретом;

- **Subscriptions** — чаты × события × container-фильтры (дерево из `listContainers`);
- **Tasks / Unified inbox** — агрегированная лента задач из всех систем поверх `UnifiedTask`;
- **Mappings** — алиасы для `/newtask`.

8. Разделение бизнес-фич: Bot vs Mini App

Принцип: **бот** — **push** и **скорость в контексте чата**; **Mini App** — **pull** и **всё, что требует форм, списков и состояния**.

8.1 Только бот

Фича	Почему именно бот	Приоритет
Уведомления о событиях	Единственный push-канал; ядро продукта	P0
Inline-кнопки под уведомлением: «Взять в работу», «Готово», «Комментировать» (reply)	Действие за 2 сек без открытия систем; формирует привычку и retention	P0
Групповые сценарии: дайджесты, задача из reply на сообщение, автосоздание задач из группы	Mini App не работает в контексте группового чата	P1

Фича	Почему именно бот	Приоритет
Быстрый ввод: <code>/newtask</code> <code><название> #alias</code>	Быстрее любого UI для тривиального случая	P1
Inline mode (<code>@bot</code> <code><поиск></code>): поиск задачи + шаринг карточки в любой чат	Вирусный механизм: коллеги видят бота в чужих чатах	P2
Deep links из уведомлений в Mini App	Бот как роутер трафика в приложение	P0

8.2 Только Mini App

Фича	Почему именно Mini App	Приоритет
Онбординг: подключение провайдеров (токен/OAuth), визуальный выбор workspace и контейнеров	Токен в чате остаётся в истории — небезопасно; выбор из дерева вместо копирования ID	P0
Настройка подписок: события × контейнеры × чаты	Форма с чекбоксами; командами — мучение	P0
Unified inbox — агрегированная лента задач из всех систем	Главная дифференциация: не «ещё один нотификатор», а единая точка входа; с двумя провайдерами в Phase 1 демонстрируется сразу	P1

Фича	Почему именно Mini App	Приоритет
Просмотр задач: списки, поиск, фильтры, полная карточка, канбан	Требует UI со скроллом и состоянием	P1
Монетизация: тарифы, оплата (Telegram Stars), апселлы	В чате продавать неудобно; Stars — нативный платёжный путь	P2
Тонкие настройки: quiet hours, шаблоны сообщений, батчинг дайджестов, язык	Редко используемые формы	P2

8.3 Серая зона — в обоих, разной глубины

Фича	Бот (быстрая версия)	Mini App (полная версия)
Карточка задачи	<code>/task <id></code> : сводка + inline-кнопки	Описание, история, вложения, сабтаски
Создание задачи	Название + alias контейнера	Все поля: приоритет, дедлайн, ассайни, вложения
Комментарии	Reply на уведомление / <code>/comment</code>	Тред целиком

8.4 Логика приоритизации (solo-разработка)

- 1. **Wedge (Phase 1):** бот-уведомления из двух систем + inline-действия — привлечение и retention; два адаптера сразу фиксируют правильную абстракцию.

2. **Снятие трения (Phase 2):** Mini App с онбордингом и настройкой подписок — конверсия в активацию.
3. **Дифференциация (Phase 3):** unified inbox — основание для платного тарифа.

Бот остаётся тонким и дешёвым в поддержке; вся сложность UI живёт в Mini App.

9. Конфигурация

```
NODE_ENV=production
PORT=3000
APP_URL=https://api.your-domain.com
MINIAPP_URL=https://app.your-domain.com

SUPABASE_URL=...
SUPABASE_SERVICE_KEY=...      # только на бэкенде; Mini
App ходит со своим JWT

ENCRYPTION_KEY=<32-byte-hex>  # app-side AES-256-GCM

TELEGRAM_BOT_TOKEN=...
BOT_MODE=webhook               # polling — для разработки
TELEGRAM_WEBHOOK_SECRET=...    # значение secret_token
для setWebhook

REDIS_URL=redis://redis:6379   # обязателен (BullMQ)

# Дефолтные rate limits (переопределяются per-connection
# через adapter.rateLimit)
CLICKUP_RATE_LIMIT_DEFAULT=100 # rpm
WRIKE_RATE_LIMIT_DEFAULT=100   # rpm, консервативно
```

10. Безопасность

Угроза	Митигация
Токены провайдеров at rest	AES-256-GCM в приложении; ключ только в env
Подлинность вебхуков ClickUp	HMAC-SHA256 по сырому телу, X-Signature , per-webhook секрет из БД
Подлинность вебхуков Wrike	Handshake X-Hook-Secret при регистрации + HMAC-SHA256 нотификаций нашим секретом
Подлинность вебхуков Jira	HMAC-SHA256 по сырому телу, заголовок X-Hub-Signature (sha256=<hex>), наш секрет из БД
Подлинность апдейтов Telegram	secret_token + заголовок X-Telegram-Bot-API-Secret-Token
Auth Mini App	Валидация initData + короткоживущий JWT
Изоляция данных	App-level фильтрация по user_id; RLS через JWT-claim для Mini App (§3.1)
Токен в истории чата	/connect удаляет сообщение; основной путь ввода — Mini App
Дубли уведомлений	Unique-констрейнт (subscription_id, dedupe_key)
Transport	HTTPS only, TLS 1.2+

11. Деплой на VPS

Единый сервер, всё в Docker Compose:

```

services:
  app:          # Integration Service + Bot (один
процесс или app + worker)
    build: .
    env_file: .env
    depends_on: [redis]
  worker:      # BullMQ worker (можно масштабировать
отдельно)
    build: .
    command: node dist/worker.js
    env_file: .env
    depends_on: [redis]
  redis:
    image: redis:7-alpine
    volumes: [redis-data:/data]
  nginx:      # reverse proxy + статика Mini App
    image: nginx:alpine
    ports: ["80:80", "443:443"]
  volumes:
    redis-data:

```

Требования и заметки:

- **HTTPS обязателен** для вебхуков Telegram и Mini App — Let's Encrypt (certbot) на nginx; поддомены `api.` (сервис) и `app.` (статика Mini App).
- nginx проксирует `/webhooks/*` и `/api/*` в app; на `app.` — статика Vite-сборки.
- БД — managed Supabase (внешняя); при желании полной самодостаточности заменяется self-hosted PostgreSQL на том же VPS — код зависит только от SQL-слоя, миграции — обычные SQL-файлы.
- Health check: `GET /health` → `{ "status": "ok", "queue": "ok", "timestamp": ... }` + restart policy в compose.
- Логи: Pino → stdout → journald/файл; ротация logrotate.
- Мониторинг-метрики: успешность доставки вебхуков per-provider, латентность event → message, остаток rate limit per-

connection, счётчик протухших токенов, глубина очереди BullMQ.

12. Roadmap

Phase 1 — Notifications, два провайдера (P0), недели 1–3

- Схема БД (включая webhooks с секретами)
- BullMQ + Redis, fast-ACK endpoint `POST /webhooks/:provider`, реестр адаптеров
- **ClickUp adapter**: verify (X-Signature), parseEvents, enrich (GET /task)
- **Wrike adapter**: handshake (X-Hook-Secret), verify, parseEvents (батчи), enrich (GET /tasks/{id})
- `/connect <provider>` (с удалением сообщения), `/subscribe`, `/unsubscribe`
- Notifier с per-chat лимитером и идемпотентностью на уровне БД
- Inline-кнопки под уведомлениями («Взять в работу» / «Готово» / «Комментировать») через unified-статусы

Phase 2 — Mini App + Task Bot (P1), недели 4–6

- Mini App: auth через initData, экраны Connections/Subscriptions (рендер из реестра адаптеров)
- `/newtask`, `/task` с inline-клавиатурой, `/comment`, `/status`, `/assign`, `/due`
- `/browse` по unified-дереву контейнеров
- Дайджест-режим для групп (батчинг при всплесках)

Phase 3 — Jira + Unified inbox, недели 7–9

- **Jira adapter** (§4.3): Basic auth credentials, admin-webhook онбординг в Mini App, разворачивание changelog, транзишены, ADF
- **Unified inbox** в Mini App (лента `UnifiedTask` из всех систем)

- Кастомные шаблоны сообщений per-subscription
- Тарифы + оплата через Telegram Stars

Интерфейс адаптера (§2.1) спроектирован под Jira **уже сейчас** (opaque credentials, `getAvailableStatuses(taskId)`, `capabilities()`, headers в `parseEvents`) — в Phase 3 не потребуется рефакторинг ядра, только новый файл-адаптер по чек-листу §4.4.

Spec version: 2.3